

Uebung_06b_Binaerzahlen_loesung

July 30, 2026

1 Übung 06b - Musterlösung

In der heutigen Übung haben Sie erneut die Möglichkeit, folgende Themen zu üben:

- Händisches Umrechnen von Binär und Hexadezimalzahlen
- Umwandeln von ASCII-Code in Text
- Umrechnen von Gleitkommazahlen im Binärsystem

2 Umrechnung zwischen Binär- und Dezimalsystem

Umrechnung von Binär $\rightarrow$ Dezimal

Wir wollen die Binärzahl

$$10101_2$$

in das Dezimalsystem umwandeln.

Rechenweg:

Jede Stelle der Binärzahl steht für eine Zweierpotenz (von rechts nach links):

Stelle:	1	0	1	0	1
Potenz:	2^4	2^3	2^2	2^1	2^0

Nun multiplizieren wir jede Stelle mit der entsprechenden Potenz von 2:

$$1 \cdot 2^4 = 160 \cdot 2^3 = 01 \cdot 2^2 = 40 \cdot 2^1 = 01 \cdot 2^0 = 1$$

Dann addieren wir die Ergebnisse:

$$16 + 0 + 4 + 0 + 1 = \boxed{21}$$

Ergebnis:

$$10101_2 = \boxed{21_{10}}$$

Umrechnung von Dezimal $\rightarrow$ Binär

Beispiel:

Wir wollen die Dezimalzahl

$$43_{10}$$

ins Binärsystem umwandeln.

Rechenweg (durch wiederholtes Teilen durch 2):

$$\begin{array}{rcl} 43 \div 2 & = & 21 \quad \text{Rest: } 1 \\ 21 \div 2 & = & 10 \quad \text{Rest: } 1 \\ 10 \div 2 & = & 5 \quad \text{Rest: } 0 \\ 5 \div 2 & = & 2 \quad \text{Rest: } 1 \\ 2 \div 2 & = & 1 \quad \text{Rest: } 0 \\ 1 \div 2 & = & 0 \quad \text{Rest: } 1 \end{array}$$

Nun lesen wir die Reste **von unten nach oben**:

$$\boxed{101011}_2$$

Ergebnis:

$$43_{10} = \boxed{101011}_2$$

Darstellung negativer Ganzzahlen (Vorzeichen und Betrag)

In dieser Übung werden negative Ganzzahlen in der **Vorzeichen-Betrag-Darstellung** dargestellt (nicht im Zweierkomplement):

- Das **erste (linke) Bit** ist das Vorzeichenbit: 0 = positiv, 1 = negativ
- Die **restlichen Bits** enthalten den Betrag der Zahl in gewöhnlicher Binärdarstellung

Beispiel: Wir wollen -112_{10} als 8-Bit-Zahl mit Vorzeichen darstellen.

1. Betrag berechnen: 112_{10} in Binär, mit den verbleibenden 7 Bit (1 Bit ist für das Vorzeichen reserviert):

$$112_{10} = 1110000_2$$

2. Vorzeichenbit ergänzen (negativ $\rightarrow$ 1):

$$\begin{array}{c} \downarrow \\ \text{Vorzeichen} \end{array} \quad \underbrace{1110000}_{\text{Betrag}} = 1111 \ 0000_2$$

Wertebereich: Mit n Bit stehen $n - 1$ Bit für den Betrag zur Verfügung, der Wertebereich reicht somit von $-(2^{n-1} - 1)$ bis $+(2^{n-1} - 1)$.

Umwandlung einer Dezimalzahl in ein 8-Bit-Gleitkommaformat (Mini-Float)

Wir wandeln die Dezimalzahl 0,75 in eine 8-Bit-Gleitkommazahl um. Dieses vereinfachte Format (Mini-Float) wird oft für Lernzwecke verwendet.

2.1 Mini-Float-Struktur (8 Bit)

Feld	Länge	Beschreibung
Vorzeichen	1 Bit	0 = positiv, 1 = negativ
Exponent	3 Bit	mit Bias = 3 (also tatsächlicher Wert = Code - 3)
Mantisse	4 Bit	mit implizitem 1. davor (Normierung)

2.2 Schritt 1: Umwandlung der Dezimalzahl in Binär

2.2.1 Nachkommateil: $0,75_{10}$

Wir wandeln den Nachkommateil durch **Multiplikation mit 2** um:

Schritt	Nachkommazahl $\cdot 2$	Ganzzahl (Bit)	Neuer Nachkommateil
1	$0,75 \cdot 2 = 1,5$	1	0,5
2	$0,5 \cdot 2 = 1,0$	1	0,0

Daraus ergibt sich:

$$0,75_{10} = 0,11_2$$

2.3 Schritt 2: Normalisierung

Wir bringen die Binärzahl in die Form:

$$1,1 \cdot 2^{-1}$$

2.4 Schritt 3: Bitfelder belegen

Komponente	Wert	Binär
Vorzeichen	Positiv $\rightarrow 0$	0
Exponent	$-1 + 3 = 2$	010
Mantisse (nach dem Komma)	$1 \rightarrow 1000$ (4 Bit, mit Nullen aufgefüllt)	1000

2.5 Ergebnis: 8-Bit-Mini-Float

0 010 1000

Bitposition	Bedeutung
0	Vorzeichen
010	Exponent
1000	Mantisse

2.6 Rückrechnung zur Kontrolle

- Exponent = 010 = 2 $\rightarrow$ tatsächlicher Exponent: $2 - 3 = -1$
- Mantisse: $1,1000_2 = 1,5$
- Wert:

$$1,5 \cdot 2^{-1} = \boxed{0,75}$$

Die Rückrechnung bestätigt: Das Ergebnis stimmt.

2.7 Merke

- **Nachkommazahlen** werden per Multiplikation mit 2 umgewandelt
- Das **erste Bit vor dem Komma wird nicht gespeichert** – es ist implizit (normiert)
- Der **Exponent** wird über einen **Bias** codiert (hier: Bias = 3)
- Die Mantisse wird auf eine feste Anzahl Bits **abgeschnitten oder gerundet**

Umwandlung einer 8-Bit-Gleitkommaformat-Zahl in eine Dezimalzahl

Feld	Bits	Bedeutung
Vorzeichen	1	negativ
Exponent	101	binär 5 $\rightarrow 5 - 3 = 2$
Mantisse	0100	ergibt $1,0100_2 = 1,25$

2.8 Schrittweise Rückrechnung

1. **Vorzeichenbit:**
 $1 \rightarrow$ Zahl ist **negativ**
2. **Exponent:**
 $101_2 = 5$, Bias = 3 $\rightarrow$ tatsächlicher Exponent:

$$5 - 3 = 2$$

3. Mantisse:

Durch die implizite führende 1 ergibt sich:

$$1,0100_2 = 1 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2} + 0 \cdot 2^{-3} + 0 \cdot 2^{-4} = 1 + 0 + 0,25 = 1,25$$

4. Gesamter Wert:

$$\text{Wert} = -1,25 \cdot 2^2 = -1,25 \cdot 4 = \boxed{-5,0}$$

2.9 Ergebnis:

Der 8-Bit-Wert 1 101 0100 entspricht der Dezimalzahl:

$$\boxed{-5,0}$$

2.10 Übersicht der Umrechnung

Bitfeld	Inhalt	Bedeutung
Vorzeichen	1	negativ
Exponent	101	2
Mantisse	0100	1,25
Wert berechnet	—	$-1,25 \cdot 2^2 = -5,0$

2.11 Aufgabenstellung

2.12 Aufgabenteil A

Wandeln Sie folgende Dezimalzahlen in Binärzahlen um. Beachten Sie dabei das gewünschte digitale Format.

Dezimalzahl	digitales Format
11110	8-Bit Darstellung ohne Vorzeichen
-52210	16-Bit Darstellung mit Vorzeichen
30010	16-Bit Darstellung ohne Vorzeichen
-11210	8-Bit Darstellung mit Vorzeichen
12810	8-Bit Darstellung mit Vorzeichen

Hinweis: Die Anzahl der Einsen in der binären Darstellung für die ersten drei Zahlen ist: 6, 4, 4.

Lösungsvorschlag

- $11110 = 0110\ 1111_2$
- $-52210 = 1000\ 0010\ 0000\ 1010_2$
- $30010 = 0000\ 0001\ 0010\ 1100_2$
- $-11210 = 1111\ 0000_2$
- nicht möglich, da mit 7 Bit (eines ist für das Vorzeichen vorbehalten) die maximale Zahl 127 ist

2.13 Aufgabenteil B

Wandeln Sie die folgenden Zahlen **in die jeweils andere Darstellung** um. Notieren Sie Ihren Rechenweg und geben Sie das Ergebnis an.

$$10110_2 = ?$$

$$110011_2 = ?$$

$$1001001_2 = ?$$

Wie viele **verschiedene Werte** lassen sich mit einer Binärzahl aus genau **8 Bits** (ohne Vorzeichen) darstellen?

Wie lautet der **kleinste** und der **größte Wert**?

3 Umrechnung zwischen Binär- und Dezimalzahlen (Ganzzahlen)

3.1 Aufgabe 1

Wandle die folgende **Binärzahl** in eine **Dezimalzahl** um:

$$10110_2$$

3.1.1 Lösung:

Wir rechnen mit Zweierpotenzen:

$$1 \cdot 2^4 = 160 \cdot 2^3 = 01 \cdot 2^2 = 41 \cdot 2^1 = 20 \cdot 2^0 = 0 \Rightarrow 16 + 0 + 4 + 2 + 0 = \boxed{22}$$

3.2 Aufgabe 2

Wandle um:

$$110011_2 = ?_{10}$$

3.2.1 Lösung:

$$1 \cdot 2^5 = 321 \cdot 2^4 = 160 \cdot 2^3 = 00 \cdot 2^2 = 01 \cdot 2^1 = 21 \cdot 2^0 = 1 \Rightarrow 32 + 16 + 0 + 0 + 2 + 1 = \boxed{51}$$

3.3 Aufgabe 3

Wandle um:

$$1001001_2 = ?_{10}$$

3.3.1 Lösung:

$$1 \cdot 2^6 = 640 \cdot 2^5 = 00 \cdot 2^4 = 01 \cdot 2^3 = 80 \cdot 2^2 = 00 \cdot 2^1 = 01 \cdot 2^0 = 1 \Rightarrow 64 + 0 + 0 + 8 + 0 + 0 + 1 = \boxed{73}$$

3.4 Wie viele Werte sind mit 8 Bit darstellbar?

Mit $n = 8$ Bits lassen sich $2^8 = 256$ verschiedene Werte darstellen.

Als vorzeichenlose (unsigned) Zahl reicht der Wertebereich von

$$0000\,0000_2 = 0_{10} \quad \text{bis} \quad 1111\,1111_2 = 255_{10}.$$

3.5 Aufgabenteil C

Wandeln Sie folgende binäre Gleitkommazahlen (8 Bit) in ihr dezimales Äquivalent um. Verwenden Sie die oben vorgestellte Umwandlung.

- 010101002
- 111000102

Welche Gleitkommazahl ist vom Betrag her die kleinste darstellbare Zahl in diesem Format? Hierzu ist es hilfreich zuerst die binäre Darstellung zu bestimmen.

Hinweis: Die beiden umzuwandelnden Zahlen liegen vom Betrag her zwischen 4 und 10. Schreibt man die Gleitkommazahl auf, so sind ab der zweiten Nachkommastelle alle Stellen gleich Null.

Lösungsvorschlag

- 0 101 01002 = 5.0
- 1 110 00102 = -9.0

Die betragsmäßig kleinste darstellbare Zahl ergibt sich bei minimalem Exponenten (000) und minimaler Mantisse (0000):

$$0\,000\,0000_2 \rightarrow 1,0000_2 \cdot 2^{0-3} = 1,0 \cdot 2^{-3} = 0,125$$

3.6 Aufgabenteil D

Bestimmen Sie die binäre Darstellung im obigen hypothetischen 8-Bit-Format folgender Gleitkommazahlen:

- 1.0
- 0.5
- 1.5

und optional auch für diese Zahlen:

- 3.0
- -2.5

Hinweis: Die Anzahl der Einsen in der binären Darstellung der oberen drei Zahlen ist: 2, 1, 3.

- 0 011 0000
- 0 010 0000
- 0 011 1000

und

- 0 100 1000
- 1 100 0100

3.7 Aufgabenteil E

Die folgende Zahlenreihe enthält **ASCII-Zeichencodes** in Dezimalschreibweise. Übersetzen Sie mit Hilfe der ASCII-Tabelle aus der Vorlesung jeden Wert in das entsprechende **ASCII-Zeichen** und bestimmen Sie den **gesamten Text**.

3.7.1 Aufgabe

Übersetzen Sie die folgende Dezimalreihe in lesbaren Text:

::: 72 97 108 108 111 44 32 87 111 114 108 100 33 :::

Lösung: Hallo, World!

3.8 Zusatzaufgaben

3.8.1 a)

Implementieren Sie die Umrechnung ganzer Zahlen zwischen Binär- und Dezimalsystem nach den obigen Algorithmen als: - Flussdiagramm - Pseudocode - Python Funktion

3.8.2 b)

Rechnen Sie die folgenden Hexadezimalzahlen in Dezimal bzw. Dezimalzahlen in Hexadezimal um

- 7E16
- CF16
- 4316

- 1510
- 10810
- 24310

3.8.3 c)

Mit der oberen hypothetischen 8-Bit-Darstellung sind nur wenige Gleitkommazahlen abbildbar. Beispielsweise kann die 0.1 in diesem Format nicht exakt dargestellt werden – Sie können es mal versuchen. Bei der Umwandlung ins binäre Format würde dann die nächstgelegene abbildbare Gleitkommazahl gewählt werden. Aber auch mit den in der Praxis eingesetzten 32-Bit-Darstellungen, wie nach dem IEEE754 Standard, kann die 0.1 nicht exakt dargestellt werden. Die beiden benachbarten und abbildbaren Gleitkommazahlen sind

$$0.0999999940395 \leq 0.1 \leq 0.10000000149$$

Hier ist die größere Zahl näher dran, so dass deren binäre Darstellung für die 0.1 gewählt werden würde.

Auch die Zahlen 0.2 und 0.3 können nicht exakt dargestellt werden. Wenn Sie nun einen Computer, welcher natürlich nur mit der binären Darstellung rechnen kann, die Summe aus 0.1 und 0.2 bilden lassen, wird das Ergebnis gleich 0.3 sein?

Lösungsvorschlag

- $7E16 = 12610$
- $CF16 = 20710$
- $4316 = 6710$
- $1510 = F16$
- $10810 = 6C16$
- $24310 = F316$

Lösungsvorschlag Die Antwort auf diese Frage lautet: Es kommt darauf an. Hierbei ist es wichtig zu verstehen, dass Gleitkommazahlen, wie die 0.1, 0.2 oder auch 0.3 nicht exakt als Gleitkommazahl im Dualsystem dargestellt werden können. Konkret für diese Zahlen heißt das, dass bei der Umwandlung in eine binäre Gleitkommazahl

- 0.1 zu 0.10000000149,
- 0.2 zu 0.20000000298 und
- 0.3 zu 0.30000001192

angenähert werden. Hinweis: Nützlich für diese Betrachtung ist ein Gleitkommazahlkonverter, wie dieser [Gleitkommazahlkonverter](#).

Addiert man nun die binären Repräsentationen von 0.1 und 0.2, ergibt das

$$0.10000000149 + 0.20000000298 = 0.3000000045$$

Die Zahl 0.3000000045 kann binär nicht exakt abgebildet werden, sondern wird als 0.30000001192 abgespeichert. Das entspricht exakt der binären Darstellung von 0.3. Damit würde in diesem Fall die Addition von 0.1 und 0.2 in diesem Zahlenformat die 0.3 ergeben.

Das muss aber nicht immer so sein. Die Addition von 1.1 und 0.2 verdeutlicht dies. Die binären Darstellungen lauten:

- 0.2 wird binär zu 0.20000000298
- 1.1 wird zu 1.10000002384

Die Summe beider lautet

$$0.20000000298 + 1.10000002384 = 1.3000000268.$$

Allerdings ist dies keine darstellbare Zahl in diesem Format und wird zu 1.30000007153 umgewandelt. Dies entspricht aber nicht der Darstellung von 1.3, welche 1.29999995232 wäre. Das bedeutet, dass bei der Berechnung obiger Summe folgende Ausgabe zu erwarten wäre

$$0.2 + 1.1 = 1.30000007153.$$

```
[4]: import numpy as np
np.set_printoptions(precision=11, floatmode='fixed')
a = np.array([0.1, 0.2, 0.3, 0, 0], dtype='float32')
a[3] = a[0] + a[1]
a[4] = (a[0] + a[1]) - a[2]
print('0.1          -> {:.11f}'.format(a[0]))
print('0.2          -> {:.11f}'.format(a[1]))
print('0.3          -> {:.11f}'.format(a[2]))
print('0.1+0.2      -> {:.11f}'.format(a[3]))
print('(0.1+0.2)-0.3 -> {:.11f}'.format(a[4]))
print()

a = np.array([1.1, 0.2, 1.3, 0, 0], dtype='float32')
a[3] = a[0] + a[1]
a[4] = (a[0] + a[1]) - a[2]
print('1.1          -> {:.11f}'.format(a[0]))
print('0.2          -> {:.11f}'.format(a[1]))
print('1.3          -> {:.11f}'.format(a[2]))
print('1.1+0.2      -> {:.11f}'.format(a[3]))
print('(1.1+0.2)-1.3 -> {:.11f}'.format(a[4]))
```

```
0.1          -> 0.10000000149
0.2          -> 0.20000000298
0.3          -> 0.30000001192
0.1+0.2      -> 0.30000001192
(0.1+0.2)-0.3 -> 0.00000000000
```

```
1.1          -> 1.10000002384
0.2          -> 0.20000000298
```

1.3 -> 1.29999995232
1.1+0.2 -> 1.30000007153
(1.1+0.2)-1.3 -> 0.00000011921